

Dato: 22/04/2026

Klasse: 3.N

Fag: Programmering B

Image Piano

Elev: Magnus Holberg Stoltenberg

Lærer: Anders Roland Pedersen (andp)

Indhold

Kort abstract	1
Problemformulering	1
Funktionsbeskrivelse	1
Dokumentation	2
Udviklingsprocessen	3
Pseudokode	3
Kode udsnit	4
Test og fejlfinding	5
Konklusion	5
Bilag	5

Kort abstract

Dette eksamensprojekt undersøger, hvordan programmering kan bruges til at skabe et interaktivt og kreativt værktøj, hvor et billede omsættes til lyd baseret på farveinformation. Programmet er udviklet i Python ved brug af Pygame-biblioteket og kombinerer hændelsesstyret interaktion, billedbehandling og lydafspilning. Projektet demonstrerer, hvordan programmering kan anvendes til at udforske et æstetisk og teknisk problemområde, og hvordan iterative udviklingsmetoder, test og dokumentation indgår i processen. Programmet er også blevet kode med en mulighed for at skifte mellem flere billeder, og generere lyd dynamisk ud fra musepositionen. Synopsen dokumenterer udviklingsprocessen, test og fejlfinding.

Problemformulering

Hvordan kan jeg udvikle et interaktivt Python-program, der omsætter farveinformation fra et billede til lyd i realtid, og hvordan kan hændelsesstyret programmering, billedbehandling og iterative udviklingsmetoder anvendes til at skabe et funktionelt og brugervenligt produkt?

Funktionsbeskrivelse

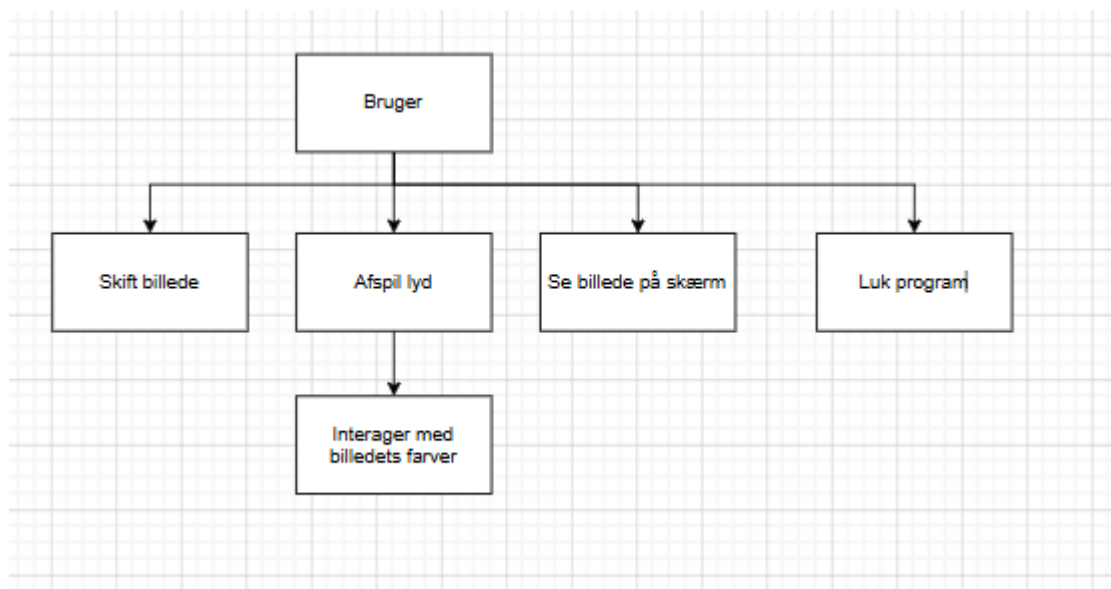
Programmet fungerer som et digitalt instrument, hvor brugeren bevæger musen hen over et billede, og farven under musemarkøren omsættes til en lyd. Og det som programmet gør, er at indlæse alle billeder fra billede mappen samt et ekstra billede som er fra en genereret nebula ude fra billede mappe, programmet giver også brugeren mulighed for at skifte mellem billeder ved at trykke på

tasterne 1–9, dog der er kun 4. Programmet afspiller en af 24 lyde afhængigt af farvens lysstyrke det justerer også automatisk vinduets størrelse efter det valgte billede. Programmet sikrer også at musepositionen altid er inden for billedets gyldige pixelområde (dog det er der lidt nogle problemer med), programmet afspiller også lyden øjeblikkeligt når musemarkøren er over en ny farve og den stopper den lyd den er i gangen med og afspiller en ny en ny lyd når den, bliver holdt over en ny farve, og programmet lukker ved at trykke på ESC. Og alt dette giver programmet en hændelsesstyret interaktion, input-output-styring, brug af eksterne biblioteker, filhåndtering, og iterativ udvikling, som alle er centrale elementer i Programmering B.

Dokumentation

Hele programmet er skrevet i Python, og det bruger Pygame til at arbejde med billeder, lyd og hændelsesstyring. Programmet fungerer ved, at det tager et billede, som brugeren har lagt i mappen `test_image_M`, og omsætter billedets farver til lyd. På den måde bliver billedet til et slags digitalt klaver eller instrument, hvor hver pixel repræsenterer en bestemt lyd afhængigt af dens lysstyrke. Når programmet starter, indlæser det alle billeder i mappen samt et ekstra billede (`output.png`). Det første billede vises i et vindue, som automatisk tilpasses billedets størrelse. Brugeren kan skifte mellem billederne ved at trykke på tasterne 1–9 (dog der er kun 4), og vinduet ændrer størrelse efter det valgte billede. Programmet kører derefter i en løkke, hvor det hele tiden registrerer musemarkørens position. Ud fra musepositionen finder programmet den pixel, markøren peger på, og læser dens farve RGB-værdier. Farvens lysstyrke beregnes som gennemsnittet af R, G og B. Denne lysstyrke bruges til at bestemme, hvilken af de 24 lydfiler der skal afspilles, så jo lysere pixel, jo højere lyd-index. Og for at undgå fejl sørger programmet for, at musepositionen aldrig går uden for billedets kanter. Hvis brugeren bevæger musen hurtigt, stopper programmet den tidligere lyd og afspiller den nye med det samme, så instrumentet føles responsivt. Og programmet kan til enhver tid lukkes ved at trykke på ESC.

Use Case Diagram:



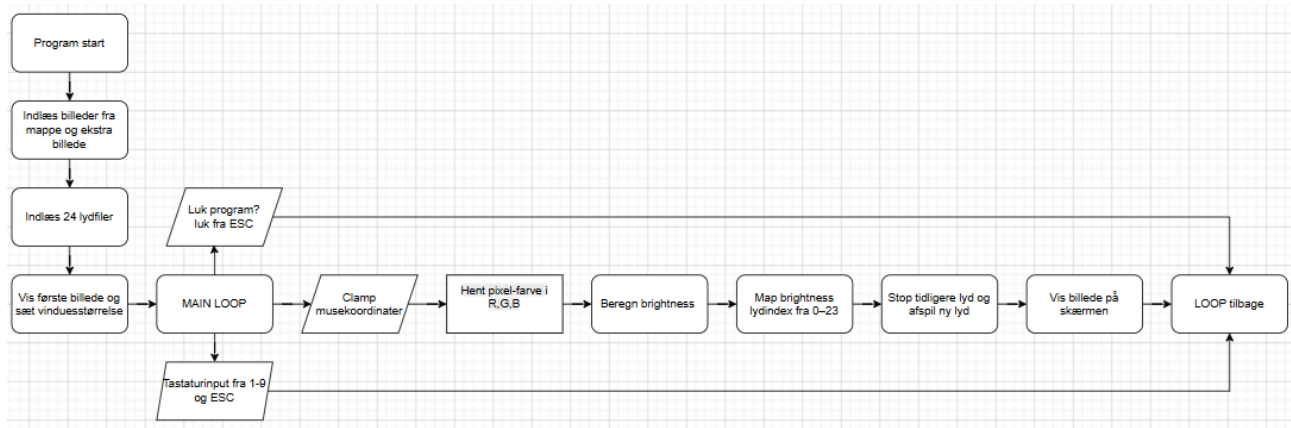
Programmet kan også beskrives i tre hovedkomponenter som er således inputlag, logiklag og outputlag.

Inputlaget er hvor programmets tastaturinput er, samt museinput og hændelsesstyringen som er via `pygame.event.get()`.

laver beregning af farve til farve lysstyrke til lyd-index, det er også her hvor programmet håndterer billedskrift, fejlhåndtering og styr lyd.

Og outputlaget er hvor programmet rendering(gengiver) billede, afspiller lyden og kontrollere vinduesstørrelsen.

Denne opdeling i inputlag, logiklag og outputlag kan ses mere tydeligt i flowchartet, hvor programmets flow og databehandling bliver illustreret trin for trin.



Udviklingsprocessen

Udviklingsprocessen af programmet startet ved at finde og lave nogle billeder som kunne testes på, efter det startede programmet med at blive lavet, og her, der startede jeg ud med at lave Pygame delen så mine billeder kunne blive vist. Efter det begyndte jeg at gøre sådan til at hver farve send en besked i konsollen nå min musemarkør var over og efter det begyndte jeg at lede efter nogle lydfiler, som jeg kunne bruge, og jeg fandt nogen, 24 for at være præcis. Efter det erstattede jeg beskederne i konsollen med lyd og satte delen op som lod mig skifte imellem forskellige billeder dog den del er der en bug i sådan fungerer ikke som den er ment at fungere. Og til sidst knyttede jeg det billede, som er blevet skabt af de andre som, der har arbejdet med cirka den samme ide som mig til mit program.

Pseudokode

Den del som jeg har lavet min Pseudokode ud af er Bilag 5.

LÆS museposition (x, y)

CLAMP x og y så de ligger inden for billedets pixelgrænser

HENT farve = pixel(x, y)

BEREGN brightness = (R + G + B) / 3

BEREGN lydIndex = brightness / 256 * 24

AFRUND lydIndex ned til nærmeste heltal

CLAMP lydIndex til intervallet 0–23

STOP tidligere lyd

AFSPIL lyd[lydIndex]

Kode udsnit

```
59 # MAIN LOOP
60 running = True
61 last_color = None
62
63 while running:
64     for event in pygame.event.get():
65         if event.type == pygame.QUIT:
66             running = False
67
68         # ESC to quit
69         if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
70             running = False
71
72         # Number keys to switch images (1-9) though there is only 4
73         if event.type == pygame.KEYDOWN:
74             if pygame.K_1 <= event.key <= pygame.K_9:
75                 num = event.key - pygame.K_1 # convert key to index 0-8
76                 if num < len(image_paths):
77                     current_index = num
78                     switch_image(current_index)
79
80     screen.blit(image, (0, 0))
81     pygame.display.flip()
```

Programmets hovedløkke styrer hele den interaktive del af systemet og kører konstant, så længe programmet er aktivt. Løkken starter med variablen `running = True`, som fungerer som en kontrolmekanisme og så længe denne variabel er sand, fortsætter programmet med at køre. Inde i løkken behandles alle hændelser, som Pygame registrerer, eksempelvis tastaturtryk eller forsøg på at lukke vinduet. Hvis brugeren klikker på ESC, registreres en QUIT-hændelse, og programmet afsluttes ved at sætte `running` til False. Derudover håndterer løkken tastaturinput fra tasterne 1–9. Når brugeren trykker på en af disse taster, omdannes tastens værdi til et tal mellem 0 og 8, som bruges som indeks

til at vælge et nyt billede fra listen over indlæste billedfiler. Hvis det valgte indeks findes i listen, kaldes funktionen `switch_image()`, som indlæser det nye billede og tilpasser vinduets størrelse. Efter hændelsesbehandlingen tegner programmet det aktuelle billede på skærmen ved hjælp af `screen.blit(image, (0, 0))`. Til sidst opdateres hele skærmen med `pygame.display.flip()`, så ændringerne bliver synlige for brugeren. Denne proces gentages mange gange i sekundet, hvilket skaber en responsiv brugeroplevelse, hvor programmet hele tiden reagerer på input og opdaterer visningen.

Test og fejlfinding

Mit program er ikke blevet testet af nogen personer andet end mig selv og ud fra de test som jeg har gjort på programmet, fandt jeg ud af at der er en bug med min kode, og jeg ved ikke vad der skaber den bug men den stopper min kode for at skifte til et nyt billede, jeg har dog mistanke på at det nok er starten i bilag 4 som kan ses med forsøg på at finde fejl, også selvom det endte forgæves, ellers kan det være noget med min mus ude af fanen, men det har jeg også prøvet og det hjalp ikke. Ellers har jeg ikke kunne finde flerger fejl i min kode.

Konklusion

Mit program viser, hvordan programmering kan bruges kreativt til at forbinde billede og lyd gennem hændelsesstyret interaktion. Og jeg har arbejdet iterativt, brugt eksterne biblioteker, udviklet et velstruktureret program og dokumenteret processen gennem modeller og kodelstykker. Og jeg vil sige at programmet opfylder kravene i projektoplægget og demonstrerer centrale faglige mål for Programmering B.

Bilag

Bilag 1: <https://github.com/sfzinstruments/SalamanderGrandPiano> (Linket til kinwie fra GitHub, hvor jeg fik musikken/lyden).

Bilag 2:

```

1  import pygame
2  import os
3
4
5  IMAGE_FOLDER = "test_image_M"          # folder with your images
6  EXTRA_IMAGE = "output.png"            # extra image outside the folder
7  SOUND_FOLDER = "sound_to_collder_M"    # folder with Sound1.wav to Sound24.wav
8
9
10 pygame.init()
11 pygame.mixer.init()
12
13
14 # LOAD ALL IMAGES
15 image_paths = []
16
17 # Load images from folder
18 for f in os.listdir(IMAGE_FOLDER):
19     if f.lower().endswith((".png", ".jpg", ".jpeg", ".bmp")):
20         image_paths.append(os.path.join(IMAGE_FOLDER, f))
21
22
23 # Load extra image if it exists
24 if os.path.exists(EXTRA_IMAGE):
25     image_paths.append(EXTRA_IMAGE)
26
27 if not image_paths:
28     raise Exception("No images found in test_image_M or extra image missing!")
29

```

Bilag 3:

```

29
30 # Load the first image
31 current_index = 0
32 image = pygame.image.load(image_paths[current_index])
33
34 width, height = image.get_size()
35 screen = pygame.display.set_mode((width, height))
36 pygame.display.set_caption("Image Piano")
37
38
39 # LOAD 24 SOUNDS (from Sound1.wav to Sound24.wav in sound_to_colleer_M)
40 sounds = []
41 for i in range(1, 25): # 1 to 24
42     filename = f"Sound{i}.wav"
43     path = os.path.join(SOUND_FOLDER, filename)
44     if not os.path.exists(path):
45         raise Exception(f"Missing sound file: {path}")
46     sounds.append(pygame.mixer.Sound(path))
47

```

Bilag 4:

```

48
49 # FUNCTION: SWITCH IMAGE (Dunno if it's a bug, but this part doesn't really function, you can still use it, but doesn't function)
50 def switch_image(index):
51     global image, width, height, screen
52     print("Switching to:", image_paths[index]) # DEBUG
53     image = pygame.image.load(image_paths[index])
54     width, height = image.get_size()
55     print("Image size:", width, "x", height) # DEBUG
56     screen = pygame.display.set_mode((width, height))
57
58
59 # MAIN LOOP
60 running = True
61 last_color = None
62
63 while running:
64     for event in pygame.event.get():
65         if event.type == pygame.QUIT:
66             running = False
67
68         # ESC to quit
69         if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
70             running = False
71
72         # Number keys to switch images (1-9) though there is only 4
73         if event.type == pygame.KEYDOWN:
74             if pygame.K_1 <= event.key <= pygame.K_9:
75                 num = event.key - pygame.K_1 # convert key to index 0-8
76                 if num < len(image_paths):
77                     current_index = num
78                     switch_image(current_index)
79
80     screen.blit(image, (0, 0))
81     pygame.display.flip()
82

```

Bilag 5:


```
83     x, y = pygame.mouse.get_pos()
84
85     # Clamp mouse position to valid pixel range (Meant to help with the "bug" still doesn't help)
86     clamped_x = max(0, min(width - 1, x))
87     clamped_y = max(0, min(height - 1, y))
88
89     color = image.get_at((clamped_x, clamped_y))[:3]
90
91     if color != last_color:
92         last_color = color
93
94         # Brightness to sound index
95         brightness = sum(color) / 3
96         index = int(brightness / 256 * 24)
97         index = max(0, min(23, index))
98
99         # Stop previous playback and play immediately
100        sounds[index].stop()
101        sounds[index].play()
102
103    pygame.quit()
104
```